

It's a feature not a bug

Making sure code
you didn't write does
what you meant

Andrew Stellman

O'Reilly Author
SVP • Principal Engineer
AI-Driven Development
Quality Playbook
andrewstellman.com

LATEST: 10.17

UPDATE

CHANGES IN VERSION 10.17:
THE CPU NO LONGER OVERHEATS
WHEN YOU HOLD DOWN SPACEBAR.

COMMENTS:

LONGTIMEUSER4 WRITES:
THIS UPDATE BROKE MY WORKFLOW!
MY CONTROL KEY IS HARD TO REACH,
SO I HOLD SPACEBAR INSTEAD, AND I
CONFIGURED EMACS TO INTERPRET A
RAPID TEMPERATURE RISE AS "CONTROL".

ADMIN WRITES:
THAT'S HORRIFYING.

LONGTIMEUSER4 WRITES:
LOOK, MY SETUP WORKS FOR ME.
JUST ADD AN OPTION TO REENABLE
SPACEBAR HEATING.

EVERY CHANGE BREAKS SOMEONE'S WORKFLOW.

I vibe-coded a bus tracker app. It sent me to the wrong stop.

It took Claude Code about two minutes to get the app working. I configured my stop and how long it takes me to walk there, and it told me when to leave. When I walked to the stop in Brooklyn, the bus showed up exactly the right time... but it was headed the wrong direction down 7th Avenue.

*It was pretty obvious what went wrong.
But why did the AI make such a basic mistake? Whose fault was it?*



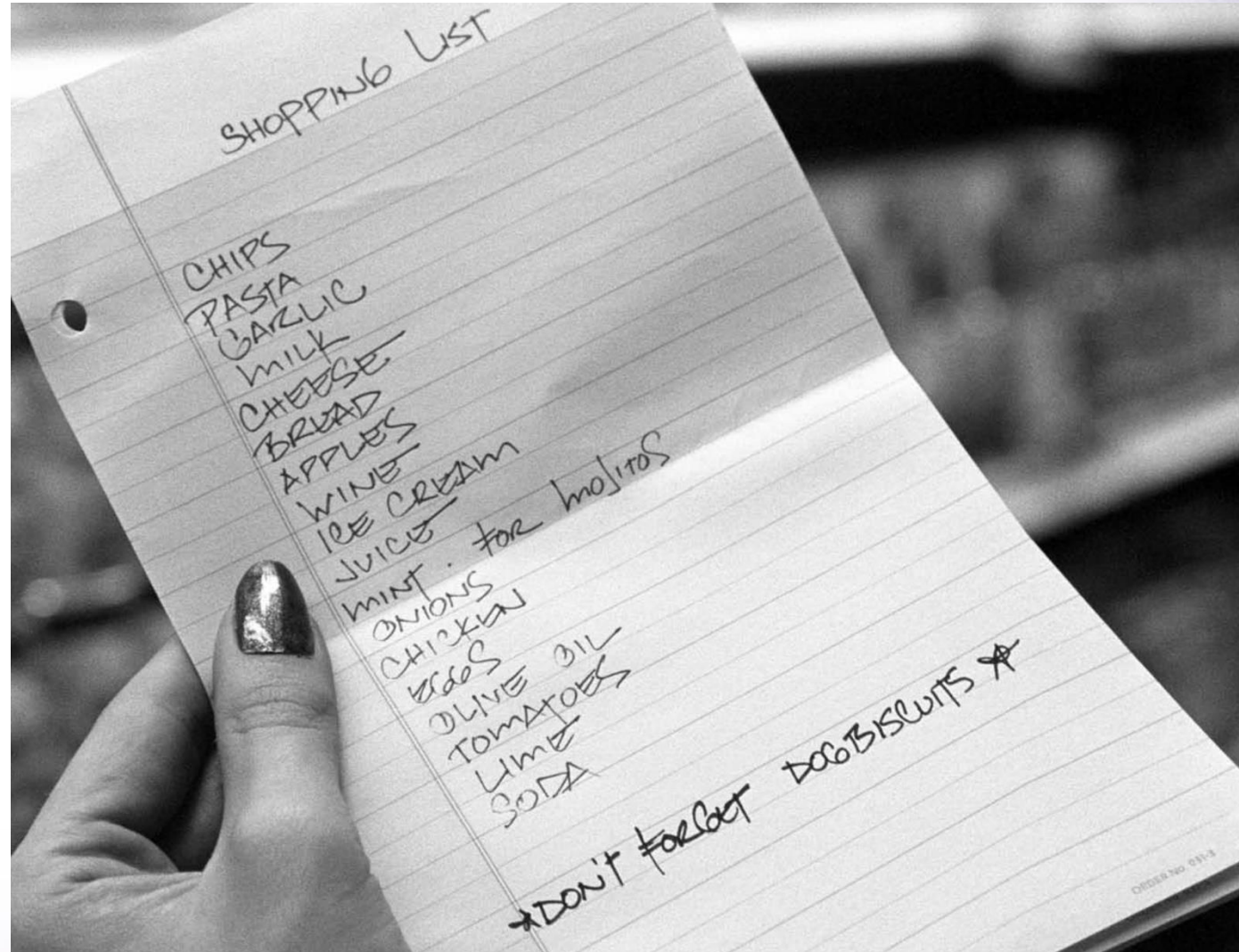
Specification, delegation and verification

I learned a lot about code quality from managing development teams, especially large groups. I couldn't review everyone's code, and I didn't want to micromanage people. So how did I make sure they met my high quality standards?

Writing down what you need is specification.

Asking someone else to shop is delegation.

Checking the bag when they get home is verification.



Over fifty years later, and we're still asking for the wrong thing



1968 NATO Conference, Software Components Lecture - M.D. McIlroy

SOFTWARE ENGINEERING

Report on a conference sponsored by the
NATO SCIENCE COMMITTEE
Garmisch, Germany, 7th to 11th October 1968

Chairman: Professor Dr. F. L. Bauer
Co-chairmen: Professor L. Bolliet, Dr. H. J. Helms

Editors: Peter Naur and Brian Randell

January 1969

Explaining what we want to build has always been one of the hardest parts of software engineering... and now we're facing it again with AI.

Writing down requirements isn't always enough

Mars Climate Orbiter, 1999

the software supplied

pound-force seconds

the specification required

newton-seconds

NASA lost the spacecraft on arrival at Mars because the ground software supplied thruster data in pound-force seconds when the interface specification required newton-seconds. Nobody checked that the software followed the requirement. So what does it mean for software to have quality in the first place?



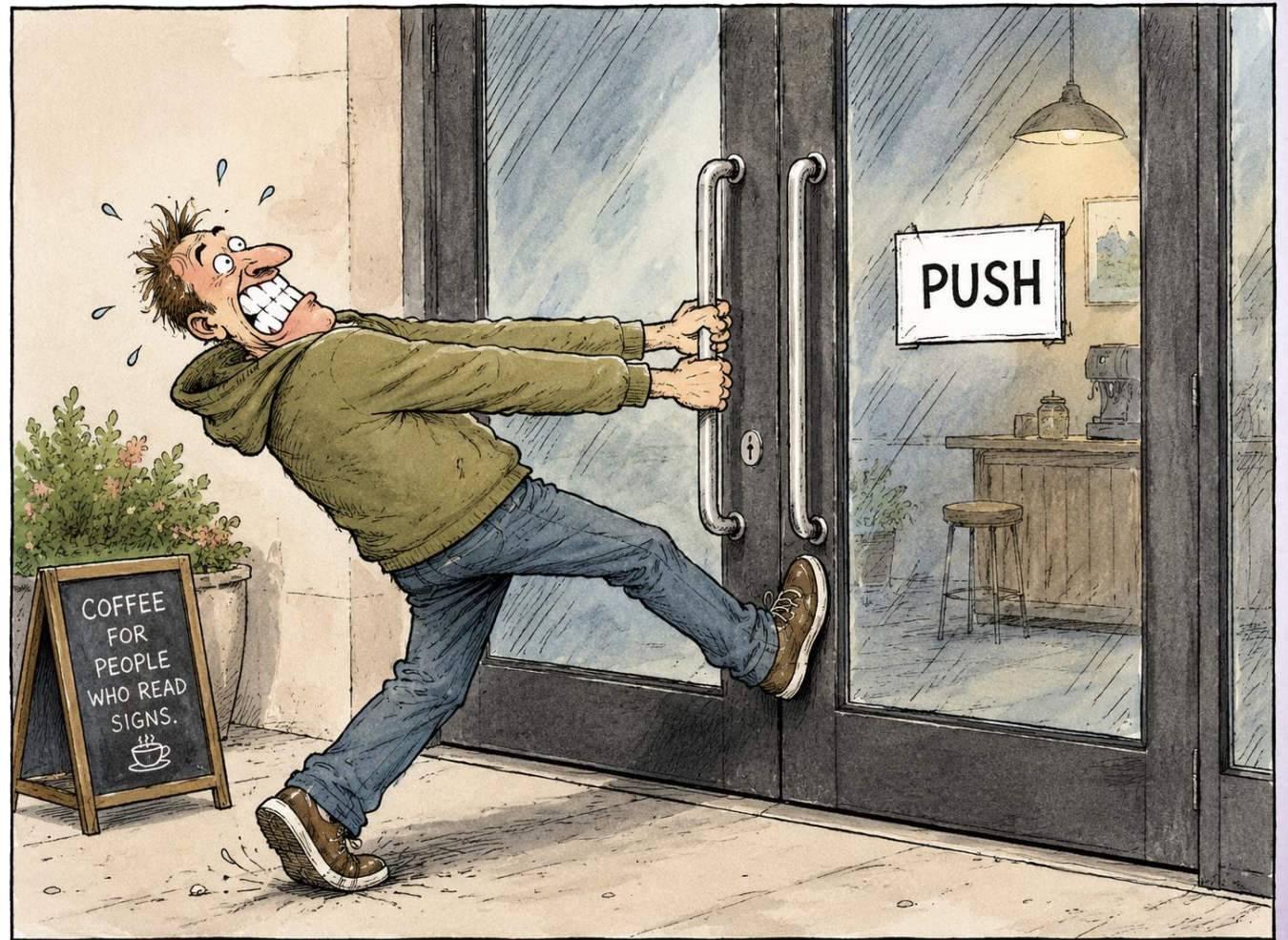
NASA/JPL artist's depiction

The AI did exactly what you said.

It just wasn't what you meant.

Have you ever built **exactly** what someone asked for, and then watched them realize it wasn't what they needed?

Have you watched an AI do the same thing, except much, much faster?



AI-generated image (and the AI got it right the first time!)

THE SIGN SAID PUSH. THE HANDLE SAID PULL.

Quality engineering predates software



Deming

1950

Lectures in Japan

**Build quality in from the start,
don't inspect it in at the end.**



Juran

1951

Quality Control Handbook

**Quality means being
fit for use.**



Crosby

1979

Quality Is Free

**Quality means meeting
the requirements.**

Quality engineering checks that the requirements, code,
and tests reflect our **intent**.

When an agent does that work, how do we check that the **intent** survived?

The requirements, code, and tests can agree while losing the original intent.

How the bus app could pass the wrong test

Original **intent**: show up just in time for a B69 bus toward Kensington.

Requirement

Show arrivals
at my stop.



Implementation

Fetch and display
those arrivals.



Test

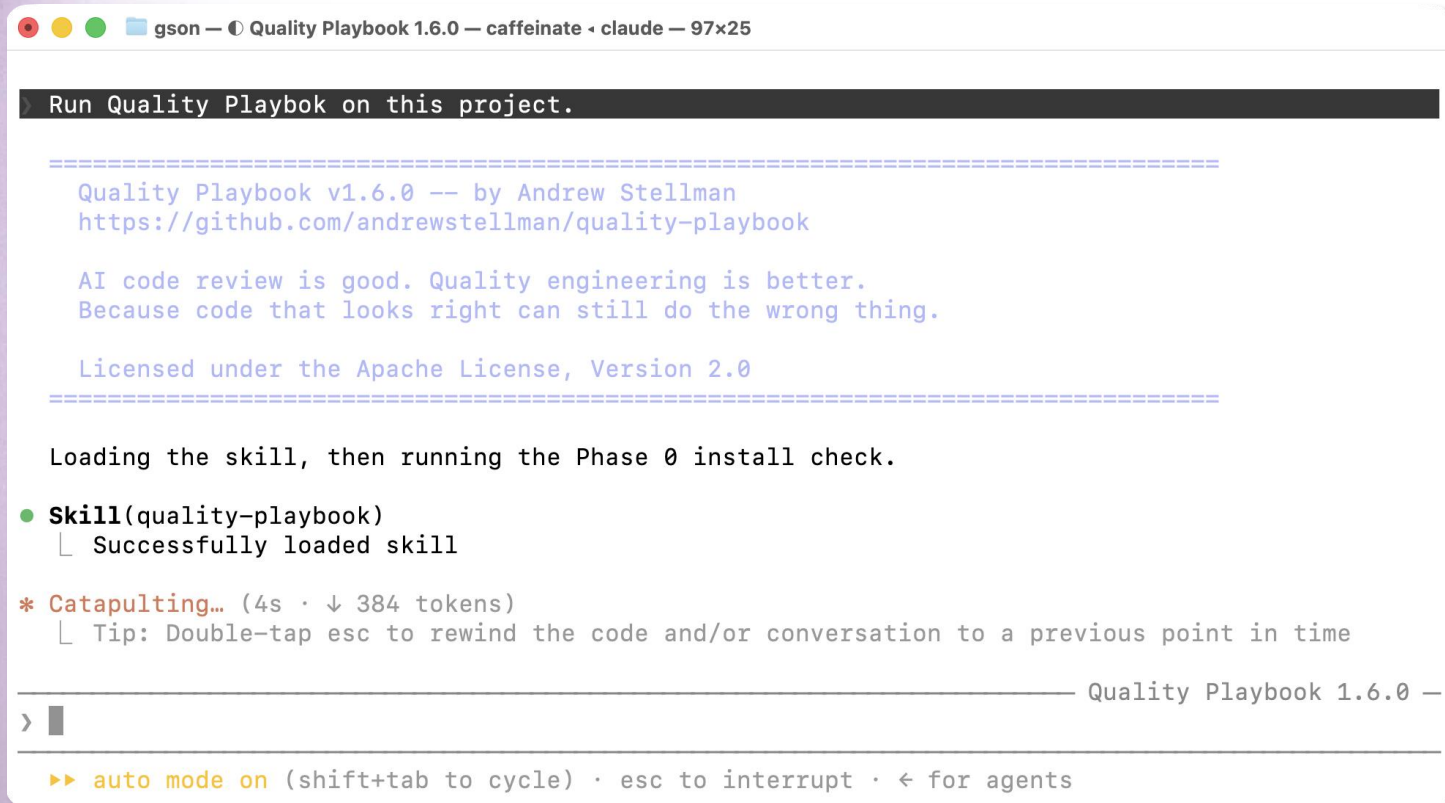
Displayed arrivals
match returned data.

✓ Passed

I showed up just in time for the bus going toward Kensington,
but the one that arrived was going toward downtown Brooklyn.

✗ Failed

Quality Playbook checks code against what it's supposed to do.



```
gson — Quality Playbook 1.6.0 — caffeinate < claude — 97x25

> Run Quality Playbok on this project.

=====
Quality Playbook v1.6.0 -- by Andrew Stellman
https://github.com/andrewstellman/quality-playbook

AI code review is good. Quality engineering is better.
Because code that looks right can still do the wrong thing.

Licensed under the Apache License, Version 2.0
=====

Loading the skill, then running the Phase 0 install check.

● Skill(quality-playbook)
  └ Successfully loaded skill

* Catapulting... (4s · ↓ 384 tokens)
  └ Tip: Double-tap esc to rewind the code and/or conversation to a previous point in time

----- Quality Playbook 1.6.0 -----
> █

>> auto mode on (shift+tab to cycle) · esc to interrupt · ← for agents
```

Quality Playbook is an open-source skill for Claude Code, Cursor and Copilot.

It reconstructs **intent** from formal and informal documentation, including project discussions and users' reports.

It turns that evidence into requirements, checks the code against them, and confirms findings with failing tests.

I review every finding before anything goes upstream.

AI is really good at writing “correct” code that does the wrong thing.

Checking the code against intent finds bugs that have been there for years.

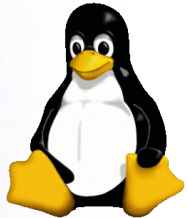
Hundreds of findings across dozens of projects tested.



Google Gson

A duplicate key slipped through when its first value was null.

PR #3006



Linux zram

A mistyped setting made recompression process every slot.

2f529e73

Linux virtio-pci

The driver handled an interrupt, then told the kernel it hadn't.

93fa0945

17 years

virtio defect: 2009 to 2026

Other highlights: calibre (PR #3310) and aiohttp (PR #13870).

I'm submitting these one at a time, after reproducing each bug and reviewing the fix.
(Maintainers have enough AI slop to deal with.)

Gson accepted duplicate keys when the first value was null.

Gson's map adapter should reject duplicate keys so applications don't silently accept ambiguous data.

Before the fix: both inputs contain the key "a" twice, which Gson's map adapter should reject. Quality Playbook recovered the **intent** behind Gson's null handling from GitHub issues reported by its users.

`{"a": 1, "a": 2}` ✓ Rejected

`{"a": null, "a": 1}` ✗ Accepted

Quality engineering means listening to your users. Their bug reports revealed real requirements nobody had written down yet, which let Quality Playbook find a bug that (amazingly) was still in the code.

Help I'm Looking For.



Try Quality Playbook on a project you know well.

Tell me what it misses or misunderstands about your **intent**.

Help me evaluate it against real requirements.

I'd like to compare its findings with bugs your team already knows.

github.com/andrewstellman/quality-playbook

When an agent writes the code, we still have to check that the original intent made it into the software and that our tests verify it.

Quality engineering checks whether the requirements capture what people need, whether the code meets those requirements, and whether the tests check those expectations. Project discussions and users' reports help us discover what we missed.

Before we call it “user error,” we owe our users that investigation.

Thanks for listening!

